Language Modeling Across Script Variants: The Case of Japanese

Go Inoue,[1,2] Nizar Habash,[1,2] Timothy Baldwin[1]
[1]Mohamed bin Zayed University of Artificial Intelligence
[2]New York University Abu Dhabi
{go.inoue,timothy.baldwin}@mbzuai.ac.ae
nizar.habash@nyu.edu

Abstract

We investigate the effect of script variants on language modeling in Japanese, which uniquely combines logographic (*kanji*) and phonetic (*kana*) writing systems. Motivated by educational curricula in which learners are gradually exposed to increasing numbers of logographic characters, we simulate this staged exposure during training to study its effect on model behavior. We construct multiple versions of a corpus with controlled script transformations, including fully transliterated kana text and intermediate variants corresponding to curriculum-based kanji levels, and train language models under different schedules that vary the timing and progression of script exposure during pre-training. In addition, we propose a cross-script evaluation framework using multiple versions of a minimal-pair benchmark with systematically varied kanji usage to assess the robustness of language models to script variation. Our results show that training on a mixture of two extreme script variants improves robustness across script conditions, whereas curriculum-based progression does not consistently improve performance.[1]

1 Introduction

Writing systems vary substantially across the world's languages, ranging from phonographic writing systems (e.g., alphabets and syllabaries) to logographic systems such as Chinese characters.[2] While many languages are predominantly written using a single script, Japanese uniquely combines multiple script types within a single language, including logographic *kanji* and phonetic *kana* (Coulmas, 2002). Table 1 illustrates how the same sentence can be represented with varying levels of script complexity. Importantly, kanji expressions can be transliterated into corresponding phonetic kana representations according to curriculum-based kanji levels while preserving the underlying semantics and grammatical structure.[3]

Raw	日本語のテキストを解析する。
Grade 2	日本語のテキストをかいせきする。
Kana-only	にほんごのテキストをかいせきする。
	nihongo-no tekisuto-wo kaiseki-suru.
	Japanese.language-GEN text-ACC analyze-do.
	"Analyze Japanese text."

Table 1: Example of script variations in Japanese. The same sentence can be represented with different levels of logographic characters (*kanji*) while preserving its semantic and grammatical content. For Grade 2, tokens containing out-of-level kanji are fully transliterated into their phonetic kana representations.

In Japanese literacy education, children first learn the phonetic syllabaries (*kana*; 92 basic characters), followed by the gradual introduction of logographic *kanji* throughout elementary school and beyond. The Japanese elementary school curriculum organizes 1,026 kanji characters (kyōiku "educational" kanji) into six grade levels, through which students are exposed to progressively larger and more complex subsets of kanji characters. Beyond elementary school, students continue learning additional characters to achieve literacy in the full set of 2,136 officially designated jōyō "regular-use" kanji throughout secondary education. As a result, Japanese literacy acquisition naturally involves staged exposure to increasingly complex script variants, beginning with fully phonetic representations and gradually transitioning toward standard mixed-script text with progressively richer kanji usage. Motivated by this staged exposure, we study how controlled progression of script exposure during pre-training affects language model learning and cross-script generalization.

[1]We release our code at https://github.com/mbzuai-nlp/lm_across_scripts.

[2]See Daniels and Bright (1996) for further discussion.

[3]While related to code-switching at the surface level, the linguistic content in this setting remains monolingual, and only the script representation varies.

Specifically, we construct multiple script variants of Japanese text with controlled levels of kanji usage, including fully kana-transliterated text and intermediate curriculum-based variants. Using these script-transformed corpora, we design pre-training schedules that vary the timing and progression of script exposure during training. To evaluate model robustness across script variants, we additionally construct script-transformed versions of JBLiMP (Someya and Oseki, 2023), a Japanese minimal-pair benchmark for evaluating linguistic generalization. Our experiments show that training on a mix of two extreme script variants improves script robustness, whereas curriculum progression does not consistently improve performance.

Our **contributions** are as follows:

- We provide a systematic empirical evaluation of script variation in Japanese language modeling, where the same linguistic content can be represented with varying levels of script complexity based on the Japanese educational curriculum.
- We explore a wide range of curriculum schedules for language model pre-training that vary the timing and progression of script exposure during pre-training.
- We propose a cross-script evaluation framework based on script-transformed JBLiMP variants for evaluating robustness and cross-script generalization across different levels of kanji usage.

2 Related Work

Text processing for Japanese. Japanese text processing conventionally begins with word segmentation through morphological analysis, given the absence of explicit whitespace delimiters between words. Major morphological analyzers adopt a lattice-based approach, in which candidate token sequences are generated from a morphological dictionary and the optimal segmentation path is selected using statistical scoring models (Kudo et al., 2004; Kudo, 2005; Morita et al., 2015; Tolmachev et al., 2018). The choice of dictionary also affects morphological analysis, ranging from relatively coarse-grained lexicons such as IPAdic (Asahara and Matsumoto, 2003; Sato et al., 2017) to linguistically motivated fine-grained lexicons such as UniDic (Den et al., 2007). These dictionaries provide rich linguistic annotations for each entry, including part-of-speech information, inflectional features, lemmas, and kana readings. In our work, we use dictionary-provided kana readings obtained through morphological analysis to construct controlled kana-transliterated variants of Japanese text.

Curriculum learning. Curriculum learning refers to training strategies that organize training examples according to some notion of difficulty or progression (Bengio et al., 2009). Such approaches have been widely explored in language model pre-training, although their effectiveness remains mixed and research beyond English is still limited (Wilcox et al., 2024). Prior work has investigated a range of curriculum strategies, including age-based data ordering (Feng et al., 2024), lexical frequency-based curricula (Borazjanizadeh, 2023; Diehl Martinez et al., 2023), syntactic complexity (Mi, 2023; Oba et al., 2023; Güven et al., 2025), progressive vocabulary expansion (Diehl Martinez et al., 2023; Edman and Bylinina, 2023; Thoma et al., 2023), and model-based approaches (Schoenegger et al., 2025; Feng et al., 2025). Broadly speaking, recent work has explored how the ordering and composition of training data influence language model generalization and learning dynamics. In contrast, we study curricula over script complexity in Japanese by systematically varying the degree of kanji usage according to the Japanese educational curriculum while preserving the underlying semantics and grammatical structure.

Evaluation. Minimal-pair evaluation enables systematic analysis of linguistic generalization in language models through probability comparisons between minimally different grammatical and ungrammatical sentence pairs (Warstadt et al., 2020; Hu et al., 2020; Gauthier et al., 2020). This evaluation framework has been widely adopted in studies of sample-efficient language modeling, including recent BabyLM challenges (Warstadt et al., 2023; Hu et al., 2024; Charpentier et al., 2025). For Japanese, Someya and Oseki (2023) introduced JBLiMP, a Japanese adaptation of BLiMP (Warstadt et al., 2020) that covers major syntactic and grammatical phenomena in Japanese. In this work, we further construct script-transformed variants of JBLiMP with systematically varied levels of kanji usage, enabling controlled evaluation of cross-script generalization and robustness across script variants in Japanese.

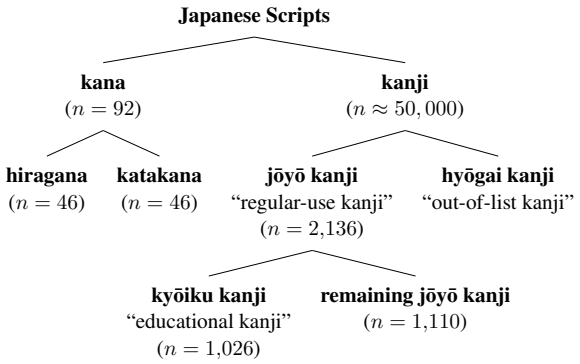


Figure 1: Overview of Japanese script systems and their character inventories.

3 The Japanese Writing System

Script variants. The Japanese writing system uses a combination of phonetic syllabaries collectively referred to as ***kana***, consisting of *hiragana* and *katakana*, each with 46 basic characters, and logographic characters (***kanji***), of which more than 50,000 are listed in dictionaries (Morohashi, 2021). Kanji are primarily used to represent lexical content such as nouns, verb stems, and adjective stems, while hiragana is commonly used for grammatical morphology and function words. Katakana is mainly used for foreign loanwords, onomatopoeia, and named entities. In typical Japanese text, writing primarily relies on the 2,136 officially designated *jōyō* “regular-use” kanji, which are intended for general everyday use. Among these, 1,026 characters are classified as *kyōiku* “educational” kanji and are taught during the six years of elementary school education, while the remaining jōyō kanji are introduced at the secondary-school level. Kanji outside the jōyō kanji are referred to as *hyōgai* “out-of-list” kanji, which occur less frequently and are often accompanied by pronunciation glosses (*ruby* or *furigana*) in educational and general-audience texts. Figure 1 summarizes the organization of Japanese script systems and the number of characters associated with each category.

Educational curriculum. Japanese literacy education begins with the learning of *hiragana* and *katakana*, followed by the gradual introduction of *kyōiku* “educational” kanji throughout elementary school. These characters are organized into six grade levels, through which students are gradually introduced to progressively larger and more complex subsets of kanji. Table 2 summarizes the six educational levels and representative examples.

Grade	#Kanji	Example
1	80	一 (one), 人 (person), 日 (day)
2	160	国 (country), 話 (speak), 今 (now)
3	200	悪 (bad), 開 (open), 和 (peace)
4	202	愛 (love), 察 (guess), 辞 (word)
5	193	境 (boundary), 潔 (pure), 構 (construct)
6	191	臓 (organ), 脳 (brain), 警 (warn)
Total	1,026	–

Table 2: Overview of the *kyōiku* “educational” kanji curriculum used in Japanese elementary education. Students are progressively introduced to 1,026 kanji characters across six grade levels.

Raw	日本語のテキストを解析する。
Morph	日本語␣の␣テキスト␣を␣解析␣する␣。
	nihongo-no tekisuto-wo kaiseki-suru.
	Japanese.language-GEN text-ACC analyze-do.
	“Analyze Japanese text.”

Table 3: Example of Japanese *scriptio continua* and its morpheme-level segmentation. The marker ␣ indicates segmented morpheme boundaries.

Early grades focus primarily on high-frequency and visually simple characters, while later grades introduce less frequent and more structurally complex forms.

Ruby annotation. Japanese text may include phonetic annotations for kanji, known as *ruby* or *furigana*, in which smaller-sized kana are visually placed above or beside kanji to explicitly indicate their pronunciation, e.g., 日本語 (にほんご) *nihongo* “Japanese language.” In machine-readable text, ruby annotations are typically represented using markup tags that associate kanji spans with their corresponding kana readings. For example, the ruby annotation 日本語 (にほんご) is represented in HTML as `<ruby><rb>日本語</rb><rt>にほんご</rt></ruby>`, where `<ruby>` defines the overall annotation structure, `<rb>` specifies the base kanji text, and `<rt>` specifies the corresponding phonetic reading.

Scriptio continua. As illustrated in Table 3, the Japanese writing system does not use whitespace characters to explicitly mark word boundaries, a property commonly referred to as *scriptio continua*. Instead, words are written as continuous character sequences, requiring readers and computational systems to infer segmentation from orthographic, morphological, and contextual cues.

Level	Description	#Characters	%Character			%Distribution of Kanji Level								%Err.	Len.
			Kana	Kanji	Other	1	2	3	4	5	6	7	8		
0	Kana-only	201,190,097	93.0	0.0	7.0	7.3	3.5	4.7	3.5	2.5	2.6	34.6	41.3	0.03	29.9
1	Grade 1	198,343,890	91.2	1.7	7.1	99.7	0.0	0.0	0.0	0.0	0.0	0.1	0.1	0.02	29.5
2	Grade 1-2	192,597,538	87.6	5.1	7.3	45.7	54.2	0.0	0.0	0.0	0.0	0.0	0.0	0.02	28.6
3	Grade 1-3	187,082,871	83.9	8.7	7.5	32.0	39.8	28.1	0.0	0.0	0.0	0.0	0.0	0.02	27.8
4	Grade 1-4	182,637,053	80.7	11.6	7.7	27.0	33.7	23.9	15.3	0.0	0.0	0.0	0.0	0.02	27.2
5	Grade 1-5	178,393,758	77.6	14.6	7.8	23.5	29.3	22.0	13.9	11.3	0.0	0.0	0.0	0.02	26.5
6	Grade 1-6	175,471,119	75.5	16.5	8.0	21.6	27.2	20.8	13.2	11.3	5.9	0.0	0.0	0.02	26.1
7	Jōyō	170,349,317	71.6	20.2	8.2	19.5	24.2	18.6	12.1	10.4	5.4	9.8	0.0	0.01	25.3
8	Original Raw	164,728,443	65.2	26.2	8.5	18.5	24.6	18.6	11.9	9.3	5.5	10.8	0.9	0.00	24.5

Table 4: Corpus-level statistics for the script-transformed variants used in pre-training. Level 0 corresponds to a full kana transliteration, while Level 8 corresponds to the original raw corpus. We report the total number of characters in each script-transformed corpus and their script-type distributions for each variant. We also report the distribution of kanji characters by educational level. "%Err." denotes the percentage of tokens whose kanji readings could not be automatically transliterated. "Len." denotes the average number of characters per line.

4 Language Modeling Across Script Variants

We investigate how script variation during pre-training affects language model learning and cross-script generalization in Japanese. To this end, we construct multiple versions of the same text with systematically varying levels of kanji usage and design pre-training schedules that control the progression of script exposure during training.

4.1 Script Transformations

Motivation. The Japanese writing system allows multiple script realizations of the same sentence through varying combinations of kanji and kana. Motivated by the staged introduction of kanji in Japanese education, we construct multiple corpora corresponding to different levels of kanji exposure. These transformations allow us to systematically investigate how the amount of logographic information available during training affects language model learning and cross-script generalization.

Transformation variants. We construct script variants corresponding to different levels of kanji exposure based on the Japanese educational curriculum. Specifically, we define nine transformation levels $L \in \{0, \ldots, 8\}$, where Level 0 corresponds to a fully phonetic representation in which all kanji are transliterated into hiragana, and Level 8 corresponds to the original text without transliteration. Intermediate levels progressively preserve larger subsets of kanji according to the curriculum-based inventories described in Section 3. For each level, we construct two transformation variants. The first is a *kana-transliterated* variant, in which out-of-level kanji are replaced with their hiragana readings. The second is a *ruby-annotated* variant, in which out-of-level kanji are preserved in the surface form while being accompanied by ruby annotations indicating their hiragana readings.

Transformation procedure. We use a widely used morphological analyzer[4] to obtain kana readings for each token. To determine which kanji characters to transliterate, we use the official level-annotated kanji lists provided by the Japanese Ministry of Education, Culture, Sports, Science and Technology. For each token, we determine whether each kanji character in the token belongs to the allowed set for the target level. If a token contains any out-of-level kanji characters, we convert the entire token into hiragana using the analyzer's dictionary-provided kana reading.[5] For ruby annotation, we follow the markup convention detailed in Section 3.

Corpus statistics. Table 4 summarizes corpus statistics across different transformation levels in our training corpus. As the source corpus, we use a randomly sampled subset of 100 million tokens from ReazonSpeech (Yin et al., 2023), a large-scale corpus of subtitled Japanese television programs. We observe that lower transformation levels con-

[4]We use MeCab (Kudo, 2005) with UniDic 3.1.0 (Den et al., 2007).

[5]We note that, if a token contains at least one out-of-level kanji character, the entire token is converted into hiragana, even when some constituent kanji belong to the permitted set. This limitation arises because the morphological analyzer provides readings at the token level and does not supply aligned per-character readings for kanji compounds. We leave finer-grained character-level transliteration based on explicit grapheme-to-phoneme alignment to future work.

Level	Description	#Tokens	#Chars	#Single-char.	%Single-char.	Avg. tokens/sent.	Chars/Token
0	Kana-only	677,912	2,013,260	93,637	13.8	10.0	3.0
1	Grade 1	680,145	1,984,707	103,284	15.2	10.1	2.9
2	Grade 1–2	680,372	1,927,333	119,136	17.5	10.1	2.8
3	Grade 1–3	680,972	1,871,504	132,520	19.5	10.1	2.7
4	Grade 1–4	680,028	1,826,700	142,448	20.9	10.1	2.7
5	Grade 1–5	678,905	1,784,319	150,659	22.2	10.1	2.6
6	Grade 1–6	677,514	1,754,943	157,557	23.3	10.0	2.6
7	Jōyō	676,526	1,703,637	175,557	25.9	10.0	2.5
0^r	Ruby-all	2,302,983	8,836,669	454,632	19.7	34.1	3.8
1^r	Ruby except Grade 1	2,093,884	7,953,750	403,767	19.3	31.0	3.8
2^r	Ruby except Grade 1–2	1,721,862	6,355,164	331,524	19.3	25.5	3.7
3^r	Ruby except Grade 1–3	1,422,010	5,027,235	282,658	19.9	21.1	3.5
4^r	Ruby except Grade 1–4	1,210,366	4,079,773	253,842	21.0	17.9	3.4
5^r	Ruby except Grade 1–5	1,053,005	3,344,517	240,204	22.8	15.6	3.2
6^r	Ruby except Grade 1–6	933,132	2,802,819	226,009	24.2	13.8	3.0
7^r	Ruby except Jōyō	697,879	1,737,243	199,038	28.5	10.3	2.5
8	Original Raw	677,641	1,647,385	196,914	29.1	10.0	2.4
MeCab (UniDic 3.1.0)		999,962	1,647,385	528,381	52.8	14.8	1.6

Table 5: Tokenization statistics on the development set across script variants using the shared BPE tokenizer with a vocabulary size of 32,000, trained jointly on the original raw corpus and its fully kana-transliterated counterpart. Levels 0–7 denote kana-transliterated variants with progressively more kanji retained, superscript r denotes the corresponding ruby-annotated variants, and level 8 denotes the original text.

tain substantially fewer kanji characters and correspondingly larger proportions of hiragana due to the transliteration of out-of-level kanji into phonetic form. The distribution of kanji levels also shifts systematically across transformation levels, with lower-level corpora primarily consisting of early-grade *kyōiku kanji* "educational kanji" and higher-level corpora gradually incorporating more advanced and out-of-list kanji characters. Although some kanji tokens fail to transliterate because of morphological analysis errors or missing dictionary readings, the error rate remains negligible across all variants (at most 0.03%). We also observe a monotonic increase in average sequence length at lower transformation levels, reflecting the replacement of kanji forms with longer phonetic kana sequences.

Tokenization statistics. In Table 5, we report tokenization statistics on the development set, including the total number of subword tokens and characters, the number and proportion of single-character subword tokens, the average number of subword tokens per sentence, and the average number of characters per subword token. We use a byte-pair encoding (BPE) tokenizer trained jointly on the original raw corpus and its fully kana-transliterated counterpart (see Section 5.1 for details). Because the BPE tokenizer is trained directly on unsegmented text rather than MeCab-segmented text, we report the average number of characters per subword token as a measure of tokenizer compactness, instead of the conventional word-based token fertility. For reference, we also report the token count segmented with MeCab using UniDic 3.1.0.

Across the kana-transliterated variants, the total number of BPE tokens remains nearly constant. However, transliterating more kanji substantially increases the total number of characters, from 1.6 million in the original text to 2.0 million at level 0 (full kana-transliteration). The average number of characters per BPE token increases from 2.4 to 3.0, while the proportion of single-character tokens decreases from 29.1% to 13.8%. Despite these differences in character length and token composition, the average number of BPE tokens per sentence remains stable at approximately 10 across all kana-transliterated variants. This indicates that the additional characters introduced through kana transliteration are largely absorbed into multi-character subwords, with little effect on the total token count.

In contrast, ruby annotation has a substantially greater effect on tokenization because it preserves the original kanji while adding their readings in kana. Both the token and character counts decrease progressively as ruby annotations are applied to fewer kanji, from 2.3 million tokens and 8.8 million characters at level 0^r to 0.7 million tokens and 1.7 million characters at level 7^r, approaching the corresponding counts for the original raw text. We also report subword vocabulary overlap across script variants in Appendix B (Figure 3).

Script	Type	Example Schedule	Description
Single-level	Baseline	8 8 8 8 8 8 8 8 8 8	Baseline training on original raw text
	Kana	0 0 0 0 0 0 0 0 0 0	Training entirely on level 0 (kana-only) text
	Ruby	$0^r 0^r 0^r 0^r 0^r 0^r 0^r 0^r 0^r 0^r$	Training entirely on level 0 text with ruby annotations
Two-level	Forward	0 0 0 8 8 8 8 8 8 8	Early simplified exposure with varying durations of raw text
	Backward	8 8 8 8 8 8 8 0 0 0	Late simplified exposure with varying durations of raw text
	Random	Random$(8, 0)^{10}$	Random mixing between level 8 (raw) and 0 (kana-only) variants
Multi-level	Progressive	0 1 2 3 4 5 6 7 8 8	Gradually increasing levels following the educational curriculum
	Reverse	8 8 7 6 5 4 3 2 1 0	Gradually decreasing levels to study curriculum directionality

Table 6: Summary of pre-training schedule families. Schedules are represented as sequences of script levels assigned to successive training epochs, where each symbol corresponds to one epoch. Levels 0–7 denote curriculum-based script variants with progressively increasing kanji usage, and level 8 denotes the original raw corpus with no transliteration applied. Superscript r indicates the ruby-annotated variant.

4.2 Pre-training Schedules

Motivation. Inspired by the staged introduction of kanji in Japanese literacy education, we investigate whether the timing and progression of script exposure during pre-training affect language model learning. Rather than training exclusively on standard orthography, we expose models to different sequences of script variants across training epochs, ranging from fully kana-transliterated text to raw text. These schedules enable us to study several questions: (a) whether exposure to different script variants improves robustness across evaluation conditions with varying levels of kanji usage; (b) whether gradually increasing script complexity following the Japanese educational curriculum yields improvements over simpler training schedules; and (c) how the timing and ordering of script exposure affect adaptation during training.

Pre-training setup. Following recent work on cognitively motivated language model training (Choshen et al., 2026), we study script curricula under a constrained pre-training setup. Specifically, under a fixed training budget of 10 epochs, we define a schedule as a sequence of script levels assigned to successive epochs. All schedules are trained using the same model architecture and optimization settings, differing only in the script representation presented during each epoch.

Schedule families. We organize pre-training schedules into three schedule families based on the number and progression of script levels used during training, as summarized in Table 6. *Single-level schedules* train on a single script level throughout all epochs, including the raw-text baseline, the kana-transliterated variant, and the ruby-annotated variant. *Two-level schedules* expose models to both kana-transliterated and raw-text variants under different exposure orderings. Forward schedules begin with fully kana-transliterated text and later transition to raw text with varying epoch allocations, enabling us to study the learning dynamics of models trained on two script variants of the same text. Conversely, backward schedules begin with raw text and later transition to kana-transliterated text to control for the timing of script exposure. We additionally include a randomized schedule in which, for each training example in every epoch, we randomly select either its kana-transliterated (level 0) or raw-text (level 8) variant, each with equal probability. Finally, *multi-level schedules* progressively vary script complexity across multiple kanji levels. Progressive curricula gradually increase script complexity following the Japanese educational curriculum, while reverse curricula progressively decrease script complexity to study the effect of curriculum directionality.

5 Experiments

We train language models under controlled curriculum settings with varying levels of kanji exposure as described in Section 4. We evaluate models using a minimal-pair benchmark across multiple script variants to analyze robustness to script variation.

5.1 Experimental Setup

Data. As pre-training data, we use Reazon-Speech (Yin et al., 2023), a large-scale corpus of subtitled Japanese TV shows. We choose TV subtitles over child-directed speech datasets such as CHILDES (MacWhinney, 2000) because of their limited data size, and also based on the advice of Feng et al. (2024), who recommend more diverse datasets than homogeneous child-directed speech datasets. We randomly sample up to 100 million

tokens of transcribed text for training purposes. Token counts are computed using MeCab (Kudo, 2005) with UniDic 3.1.0 (Den et al., 2007). Additionally, we sample held-out validation and test sets of 1 million tokens each.

Model. We use the decoder-only GPT-2 Small architecture with 124M parameters (Radford et al., 2019), following the 2026 BabyLM Workshop (Choshen et al., 2026) setup. To maintain consistency across experiments, all models use the same tokenizer, ensuring that differences in performance arise from training strategies rather than tokenization artifacts. We train a byte-pair encoding (BPE) tokenizer (Gage, 1994; Sennrich et al., 2016) with a vocabulary size of 32,000[6] on both raw text and its fully kana-transliterated counterpart, covering all character types appearing in the transformed corpora. We also include annotation markup tokens such as `<ruby>` and `<rt>`. All experiments are conducted with three random seeds, and we report average performance across runs.

5.2 Evaluation

Minimal pair evaluation. We evaluate the linguistic generalization ability of language models using JBLiMP (Someya and Oseki, 2023), a Japanese adaptation of the BLiMP minimal-pair benchmark (Warstadt et al., 2020). JBLiMP consists of 331 minimal pairs, each comprising a grammatical sentence and its ungrammatical counterpart, covering 11 major categories of linguistic phenomena. Evaluation is conducted in a zero-shot setting, where a model prediction is considered correct if the model assigns a higher probability to the grammatical sentence within each pair.

Cross-script evaluation. Japanese sentences can be represented using multiple script forms with different levels of kanji usage while preserving their underlying meaning and grammatical structure. To evaluate robustness across such script variants, we construct transformed versions of JBLiMP using the transformation procedure described in Section 4.1. Specifically, we generate multiple JBLiMP variants across curriculum levels with systematically varied kanji usage. We manually inspected the transformed evaluation data, corrected any transliteration errors, and verified that the original meanings were preserved. For each training schedule, we report the mean accuracy across three random seeds separately at each of the nine evaluation levels (L=0–8), together with the macro-average across levels.

[6]Preliminary experiments with vocabulary sizes of 16,000, 32,000, and 50,000 showed that 32,000 yielded the best overall performance.

6 Results

6.1 Main Results

Table 7 summarizes JBLiMP accuracy across different script variants and pre-training schedules.[7] The baseline model trained exclusively on raw text performs best on the original benchmark (78.7), but drops substantially as the evaluation input becomes increasingly phonetic, all the way to 66.5 on the fully kana-transliterated evaluation set (L=0). In contrast, the fully kana-trained model exhibits the opposite trend, with relatively low performance on the original benchmark (57.5) but 75.8 on the fully kana-transliterated evaluation set. The ruby-only condition performs consistently worse than kana-only training across all evaluation variants, likely because of the additional sequence complexity introduced by annotation markup.[8] Interestingly, the randomized schedule mixing raw and fully kana-transliterated variants achieves the highest average accuracy overall (76.1) and maintains relatively stable performance across all script conditions, suggesting that exposure to two extreme variants of the same text may already provide sufficient signal for learning robust cross-script representations.[9]

6.2 Effect of Schedule

Contrary to the intuition that gradually increasing script complexity would facilitate language model training, we do not observe consistent gains from curriculum-based schedules over simpler transition or static schedules on the original JBLiMP benchmark (L=8). In particular, progressive curricula following the Japanese educational ordering of kanji exposure perform comparably to, or only marginally better than, schedules with abrupt transitions to raw text. In contrast, the reverse curriculum, which ends with fully kana-transliterated text, performs substantially worse on L=8. Forward schedules, which transition from kana-transliterated to raw text, consistently outper-

[7]Corresponding perplexity results are provided in Appendix E.1 (Table 11).

[8]We report results of ruby-annotated variants for two-level and multi-level settings in Appendix E.2 (Table 12).

[9]McNemar's tests show that the randomized schedule significantly outperforms the baseline across all kana-transliterated evaluation variants ($p < 0.05$).

Script	Type	Schedule	JBLiMP (Average Accuracy)									
			L=8	L=7	L=6	L=5	L=4	L=3	L=2	L=1	L=0	AVG
Single-level	Baseline	8 8 8 8 8 8 8 8 8 8	**78.7**	74.2	71.1	71.0	70.6	70.2	68.0	68.1	66.5	70.9
	Kana	0 0 0 0 0 0 0 0 0 0	57.5	60.8	61.3	63.2	63.5	63.2	65.8	69.5	75.8	64.5
	Ruby	$0^r 0^r 0^r 0^r 0^r 0^r 0^r 0^r 0^r 0^r$	54.3	58.2	57.3	59.2	59.3	57.9	58.9	62.2	65.1	59.2
Two-level	Forward	0 0 0 0 0 0 0 0 0 8	73.9	73.2	72.3	72.1	72.9	73.9	73.4	72.8	70.8	72.8
		0 0 0 0 0 0 0 0 8 8	74.8	74.7	73.3	73.7	74.0	75.1	74.1	73.9	70.3	73.8
		0 0 0 0 0 0 0 8 8 8	75.0	73.7	71.7	71.9	72.5	73.1	72.2	72.0	68.9	72.3
		0 0 0 0 0 0 8 8 8 8	75.8	74.7	71.9	72.0	72.4	71.8	70.4	71.4	69.5	72.2
		0 0 0 0 0 8 8 8 8 8	75.8	73.5	71.2	71.1	71.5	71.6	71.0	72.1	69.4	71.9
		0 0 0 0 8 8 8 8 8 8	76.0	74.3	71.6	70.9	71.2	70.1	68.5	69.8	68.9	71.3
		0 0 0 8 8 8 8 8 8 8	76.4	74.7	72.4	72.2	72.2	70.4	68.9	69.3	68.6	71.7
		0 0 8 8 8 8 8 8 8 8	76.7	74.2	71.4	71.3	71.1	69.5	67.7	68.1	66.1	70.7
		0 8 8 8 8 8 8 8 8 8	75.6	74.1	71.8	71.5	71.0	69.7	66.5	67.1	65.5	70.3
	Backward	8 0 0 0 0 0 0 0 0 0	56.4	59.0	59.8	62.6	63.1	63.0	65.1	68.8	74.4	63.6
		8 8 0 0 0 0 0 0 0 0	55.3	61.0	61.8	64.4	64.7	64.2	66.8	69.9	75.0	64.8
		8 8 8 0 0 0 0 0 0 0	60.0	66.2	66.6	69.0	69.2	68.5	69.4	72.9	76.5	68.7
		8 8 8 8 0 0 0 0 0 0	66.2	67.3	67.6	70.0	70.1	70.1	71.1	73.7	**77.0**	70.3
		8 8 8 8 8 0 0 0 0 0	66.8	68.7	68.4	69.6	69.9	70.1	70.8	73.1	76.5	70.4
		8 8 8 8 8 8 0 0 0 0	70.6	70.0	69.4	70.9	71.0	71.4	72.0	73.4	76.4	71.7
		8 8 8 8 8 8 8 0 0 0	72.2	71.4	70.7	72.5	72.5	72.4	72.5	74.0	75.2	72.6
		8 8 8 8 8 8 8 8 0 0	72.9	72.3	71.1	71.6	71.4	72.1	73.3	74.0	75.3	72.7
		8 8 8 8 8 8 8 8 8 0	71.1	71.7	71.5	71.8	71.6	71.4	72.3	72.9	74.6	72.1
	Random	Random(8, 0)[10]	76.2	**77.1**	**76.1**	**75.7**	**75.6**	**76.7**	**75.8**	**75.6**	75.5	**76.1**
Multi-level	Progressive	0 1 2 3 4 5 6 7 8 8	76.3	73.5	72.4	73.0	73.4	73.0	71.3	72.0	67.8	72.5
	Reverse	8 8 7 6 5 4 3 2 1 0	67.2	71.2	71.6	72.9	73.0	74.0	74.5	75.2	75.9	72.8

Table 7: Main JBLiMP accuracy results across script variants and pre-training schedules (averaged across three runs). Columns (L=8 → L=0) denote script-transformed evaluation sets with progressively reduced kanji inventories, where L=8 denotes the original benchmark (raw) and L=0 corresponds to fully kana-transliterated text. AVG denotes the macro-average accuracy across all evaluation levels. Boldface indicates the best-performing model for each evaluation condition.

form backward schedules, which transition from raw to kana-transliterated text, on the original benchmark. Within both schedule families, greater exposure to raw text is generally associated with higher accuracy. These results suggest that accuracy on the original benchmark reflects both the overall amount and the recency of exposure to raw text. We further investigate the effect of recent exposure through controlled experiments in Appendix E.3.

6.3 Robustness Across Kanji Levels

Overall, evaluation performance correlates with the script similarity between the training schedule and the evaluation condition. For example, the baseline model trained exclusively on the original raw corpus shows a strong positive correlation between evaluation level and accuracy ($r = 0.92$), indicating that performance systematically improves as more kanji are retained in the evaluation data. Conversely, the kana-only model exhibits a strong negative correlation ($r = -0.85$), achieving better performance on heavily transliterated evaluation sets. In contrast, the randomized schedule maintains an almost flat performance profile across evaluation conditions, achieving consistently high accuracy regardless of kanji level. This result suggests that continued exposure to the two maximally contrasting script variants alone may be sufficient for robust generalization across the full range of script conditions. Notably, the model receives no direct alignment between corresponding examples across the two variants and no exposure to any of the intermediate, partially transliterated levels.

7 Analysis: Learning Trajectory

To better understand how script variants influence model adaptation over the course of training, we analyze the trajectory of JBLiMP accuracy across training epochs under different pre-training schedules. Figure 2 illustrates JBLiMP accuracy trajectories for representative schedules evaluated on both the original benchmark and the fully kana-transliterated benchmark.[10]

[10]We provide trajectories for all forward and backward transition schedules in Appendix E.4 (Figure 4).

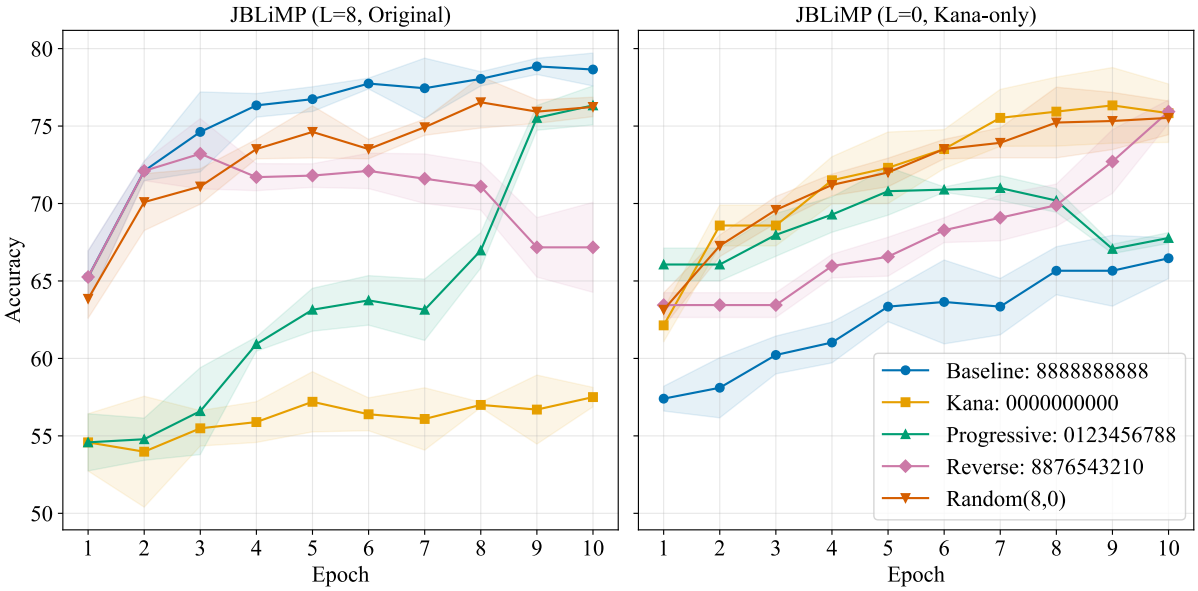


Figure 2: Learning trajectories across training epochs for representative script schedules evaluated on the original JBLiMP benchmark (L=8) and the fully kana-transliterated benchmark (L=0). We report overall accuracy at each training epoch.

On the original benchmark, the raw baseline consistently achieves the strongest performance throughout training, gradually improving from 65.3 at epoch 1 to an accuracy of 78.7 by the final epoch. The progressive curriculum initially lags behind the baseline during early epochs dominated by kana-transliterated text, but rapidly improves once higher-kanji variants are introduced later in training. In contrast, the reverse curriculum begins to degrade at epoch 4 as the training data shifts toward lower-kanji variants and never recovers to its earlier peak performance.

On the fully kana-transliterated evaluation set, both the kana-only model and the randomized schedule show similar learning trajectories, eventually converging to comparable performance. Interestingly, the reverse curriculum steadily improves throughout training as progressively simpler script variants are introduced. In contrast, the progressive curriculum begins to plateau and gradually decline once higher-kanji variants dominate the training schedule. Although the raw-only baseline also improves steadily over the course of training, it consistently remains below models exposed to kana-transliterated text.

Taken together, the curriculum trajectories suggest that progressively increasing script complexity improves performance on the original benchmark over the course of training; however, it does not consistently preserve robustness on kana-transliterated evaluation sets, limiting effective cross-script generalization throughout training.

8 Conclusion

We investigated how script variation during pretraining affects language model learning and cross-script generalization in Japanese. Our experiments showed that curriculum progression following the Japanese educational ordering of kanji exposure does not consistently yield improved performance. Instead, models trained with continual exposure to two extreme script variants of the same content, namely fully kana-transliterated and raw text, achieve the strongest overall robustness across multiple evaluation conditions. Our findings provide additional evidence for the mixed effectiveness of curriculum learning reported in prior work (Wilcox et al., 2024), extending these observations to script-based curricula in Japanese.

Limitations

Our study has several limitations. First, consistent with the BabyLM setup, our experiments are limited to the 124M-parameter GPT-2 architecture (Radford et al., 2019), trained on a single domain with a fixed tokenizer. This constraint may limit the generalizability of our findings to other model architectures, tokenization configurations, and domains. Second, our script transformation procedure relies on dictionary-based kana readings

obtained through morphological analysis and performs transliteration at the token level rather than through explicit grapheme-to-phoneme alignment. Developing accurate grapheme-to-phoneme alignment for Japanese kanji compounds is itself a challenging problem and remains outside the scope of the current work. Additionally, our evaluation focuses primarily on syntactic generalization and cross-script robustness, and does not assess downstream task performance or broader linguistic capabilities. Future work could explore these dimensions to extend our study.

Ethical Considerations

We used subtitles from Japanese TV shows, which are monitored by the Broadcasting Ethics and Program Improvement Organization. Existing biases in the data may be amplified, as our models are trained on pre-existing corpora that may contain linguistic, social, or cultural biases. The source corpus and preprocessing tools are used in accordance with their respective licenses. ReazonSpeech is distributed under the CDLA-Sharing-1.0 license, and the licenses of all data and tools are listed in Appendix D (Table 10). We used AI writing assistance within the scope of "Assistance purely with the language of the paper" described in the ACL Policy on Publication Ethics.

Acknowledgments

We acknowledge the support of the High Performance Computing Center at New York University Abu Dhabi. We thank the anonymous reviewers for their insightful and constructive comments.

References

Masayuki Asahara and Yuji Matsumoto. 2003. IPADic version 2.7. 0 user's manual. *Information Science Division, Nara Institute of Science and Technology*.

Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. 2009. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, ICML '09, page 41–48, New York, NY, USA. Association for Computing Machinery.

Nasim Borazjanizadeh. 2023. Optimizing GPT-2 pretraining on BabyLM corpus with difficulty-based sentence reordering. In *Proceedings of the BabyLM Challenge at the 27th Conference on Computational Natural Language Learning*, pages 356–365, Singapore. Association for Computational Linguistics.

Lucas Charpentier, Leshem Choshen, Ryan Cotterell, Mustafa Omer Gul, Michael Y. Hu, Jing Liu, Jaap Jumelet, Tal Linzen, Aaron Mueller, Candace Ross, Raj Sanjay Shah, Alex Warstadt, Ethan Gotlieb Wilcox, and Adina Williams. 2025. Findings of the third BabyLM challenge: Accelerating language modeling research with cognitively plausible data. In *Proceedings of the First BabyLM Workshop*, pages 399–420, Suzhou, China. Association for Computational Linguistics.

Leshem Choshen, Ryan Cotterell, Mustafa Omer Gul, Jaap Jumelet, Tal Linzen, Aaron Mueller, Suchir Salhan, Raj Sanjay Shah, Alex Warstadt, and Ethan Gotlieb Wilcox. 2026. Babylm turns 4 and goes multilingual: Call for papers for the 2026 babylm workshop. *Preprint*, arXiv:2602.20092.

Florian Coulmas. 2002. *Writing Systems: An Introduction to Their Linguistic Analysis*. Cambridge Textbooks in Linguistics. Cambridge University Press.

Peter T. Daniels and William Bright. 1996. *The world's writing systems*. Oxford University Press, New York.

Yasuharu Den, Toshinobu Ogiso, Hideki Ogura, Atsushi Yamada, Nobuaki Minematsu, Kiyotaka Uchimoto, and Hanae Koiso. 2007. The development of an electronic dictionary for morphological analysis and its application to japanese corpus linguistics. *Japanese Linguistics*, 22:101–123.

Richard Diehl Martinez, Zébulon Goriely, Hope McGovern, Christopher Davis, Andrew Caines, Paula Buttery, and Lisa Beinborn. 2023. CLIMB – curriculum learning for infant-inspired model building. In *Proceedings of the BabyLM Challenge at the 27th Conference on Computational Natural Language Learning*, pages 112–127, Singapore. Association for Computational Linguistics.

Lukas Edman and Lisa Bylinina. 2023. Too much information: Keeping training simple for BabyLMs. In *Proceedings of the BabyLM Challenge at the 27th Conference on Computational Natural Language Learning*, pages 89–97, Singapore. Association for Computational Linguistics.

Qi Feng, Yihong Liu, and Hinrich Schuetze. 2025. Your pretrained model tells the difficulty itself: A self-adaptive curriculum learning paradigm for natural language understanding. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 4: Student Research Workshop)*, pages 222–239, Vienna, Austria. Association for Computational Linguistics.

Steven Y. Feng, Noah D. Goodman, and Michael C. Frank. 2024. Is child-directed speech effective training data for language models? In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 22055–22071, Miami, Florida, USA. Association for Computational Linguistics.

Philip Gage. 1994. A new algorithm for data compression. *The C Users Journal archive*, 12:23–38.

Jon Gauthier, Jennifer Hu, Ethan Wilcox, Peng Qian, and Roger Levy. 2020. SyntaxGym: An online platform for targeted evaluation of language models. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pages 70–76, Online. Association for Computational Linguistics.

Arzu Burcu Güven, Anna Rogers, and Rob van der Goot. 2025. Do syntactic categories help in developmentally motivated curriculum learning for language models? In *Proceedings of the First BabyLM Workshop*, pages 288–300, Suzhou, China. Association for Computational Linguistics.

Jennifer Hu, Jon Gauthier, Peng Qian, Ethan Wilcox, and Roger P. Levy. 2020. A systematic assessment of syntactic generalization in neural language models. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 1725–1744, Online. Association for Computational Linguistics.

Michael Y. Hu, Aaron Mueller, Candace Ross, Adina Williams, Tal Linzen, Chengxu Zhuang, Ryan Cotterell, Leshem Choshen, Alex Warstadt, and Ethan Gotlieb Wilcox. 2024. Findings of the second BabyLM challenge: Sample-efficient pretraining on developmentally plausible corpora. In *The 2nd BabyLM Challenge at the 28th Conference on Computational Natural Language Learning*, pages 1–21, Miami, FL, USA. Association for Computational Linguistics.

Taku Kudo. 2005. MeCab: Yet Another Part-of-Speech and Morphological Analyzer.

Taku Kudo, Kaoru Yamamoto, and Yuji Matsumoto. 2004. Applying conditional random fields to Japanese morphological analysis. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, pages 230–237, Barcelona, Spain. Association for Computational Linguistics.

Brian MacWhinney. 2000. The childes project: Tools for analyzing talk: Volume i: Transcription format and programs, volume ii: The database. *Computational Linguistics*, 26(4):657–657.

Maggie Mi. 2023. Mmi01 at the BabyLM challenge: Linguistically motivated curriculum learning for pretraining in low-resource settings. In *Proceedings of the BabyLM Challenge at the 27th Conference on Computational Natural Language Learning*, pages 269–278, Singapore. Association for Computational Linguistics.

Anthony Moi and Nicolas Patry. 2023. Huggingface's tokenizers.

Hajime Morita, Daisuke Kawahara, and Sadao Kurohashi. 2015. Morphological analysis for unsegmented languages using recurrent neural network language model. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pages 2292–2297, Lisbon, Portugal. Association for Computational Linguistics.

Tetsuji Morohashi. 2021. *Dai Kanwa Jiten (Great Chinese-Japanese Character Dictionary)*, revised second edition. Taishukan Publishing Co., Ltd.

Miyu Oba, Akari Haga, Akiyo Fukatsu, and Yohei Oseki. 2023. BabyLM challenge: Curriculum learning based on sentence complexity approximating language acquisition. In *Proceedings of the BabyLM Challenge at the 27th Conference on Computational Natural Language Learning*, pages 290–297, Singapore. Association for Computational Linguistics.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI*. Accessed: 2024-11-15.

Toshinori Sato, Taiichi Hashimoto, and Manabu Okumura. 2017. Implementation of a word segmentation dictionary called mecab-ipadic-neologd and study on how to use it effectively for information retrieval (in japanese). In *Proceedings of the Twenty-three Annual Meeting of the Association for Natural Language Processing*, pages NLP2017–B6–1. The Association for Natural Language Processing.

Loris Schoenegger, Lukas Thoma, Terra Blevins, and Benjamin Roth. 2025. Influence-driven curriculum learning for pre-training on limited data. In *Proceedings of the First BabyLM Workshop*, pages 356–379, Suzhou, China. Association for Computational Linguistics.

Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany. Association for Computational Linguistics.

Taiga Someya and Yohei Oseki. 2023. JBLiMP: Japanese benchmark of linguistic minimal pairs. In *Findings of the Association for Computational Linguistics: EACL 2023*, pages 1581–1594, Dubrovnik, Croatia. Association for Computational Linguistics.

Lukas Thoma, Ivonne Weyers, Erion Çano, Stefan Schweter, Jutta L Mueller, and Benjamin Roth. 2023. CogMemLM: Human-like memory mechanisms improve performance and cognitive plausibility of LLMs. In *Proceedings of the BabyLM Challenge at the 27th Conference on Computational Natural Language Learning*, pages 180–185, Singapore. Association for Computational Linguistics.

Arseny Tolmachev, Daisuke Kawahara, and Sadao Kurohashi. 2018. Juman++: A morphological analysis toolkit for scriptio continua. In *Proceedings of the 2018 Conference on Empirical Methods in*

Natural Language Processing: System Demonstrations, pages 54–59, Brussels, Belgium. Association for Computational Linguistics.

Alex Warstadt, Aaron Mueller, Leshem Choshen, Ethan Wilcox, Chengxu Zhuang, Juan Ciro, Rafael Mosquera, Bhargavi Paranjabe, Adina Williams, Tal Linzen, and Ryan Cotterell. 2023. Findings of the BabyLM challenge: Sample-efficient pretraining on developmentally plausible corpora. In *Proceedings of the BabyLM Challenge at the 27th Conference on Computational Natural Language Learning*, pages 1–34, Singapore. Association for Computational Linguistics.

Alex Warstadt, Alicia Parrish, Haokun Liu, Anhad Mohananey, Wei Peng, Sheng-Fu Wang, and Samuel R. Bowman. 2020. BLiMP: The benchmark of linguistic minimal pairs for English. *Transactions of the Association for Computational Linguistics*, 8:377–392.

Ethan G Wilcox, Michael Hu, Aaron Mueller, Tal Linzen, Alex Warstadt, Leshem Choshen, Chengxu Zhuang, Ryan Cotterell, and Adina Williams. 2024. Bigger is not always better: The importance of human-scale language modeling for psycholinguistics.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, and 3 others. 2020. Huggingface's transformers: State-of-the-art natural language processing. *Preprint*, arXiv:1910.03771.

Yue Yin, Daijiro Mori, and Seiji Fujimoto. 2023. ReazonSpeech: A free and massive corpus for Japanese ASR. In *Proceedings of Annual Meeting of the Association for NLP*, pages 1134–1139, Okinawa, Japan.

A Data Details

We use ReazonSpeech (Yin et al., 2023), a large-scale corpus of subtitled Japanese TV shows. Table 8 shows data statistics of our dataset. We randomly sample transcribed text of up to 100 million tokens for training purposes. We count the number of tokens based on MeCab (Kudo, 2005) with UniDic 3.1.0 (Den et al., 2007). Additionally, we sample separate held-out sets of 1 million tokens each for validation and testing.

	Train	Dev	Test
#Lines	6,726,066	67,547	67,310
#Tokens	99,999,997	999,962	999,967

Table 8: Data statistics of our dataset sampled from ReazonSpeech (Yin et al., 2023).

B Pairwise Vocabulary Overlap

In Figure 3, we report the pairwise vocabulary overlap across all script variants using the Jaccard similarity between the sets of unique BPE tokens observed in each corpus. Vocabulary overlap decreases progressively as script variants become increasingly different from the original text. Neighboring script levels show consistently high overlap, whereas the fully kana-transliterated corpus shares substantially less vocabulary with the raw corpus. Ruby-annotated variants remain much closer to the original corpus, reflecting the preservation of kanji surface forms despite the addition of pronunciation annotations.

C Model Details

C.1 Hyperparameters

We generally follow the hyperparameters of the 124M-parameter GPT-2 baseline in Choshen et al. (2026). In Table 9, we provide hyperparameters used for model pre-training. We use a single NVIDIA A100 GPU (80GB) for training and testing all models. The total computational cost for our experiments is approximately 1,135 GPU hours.

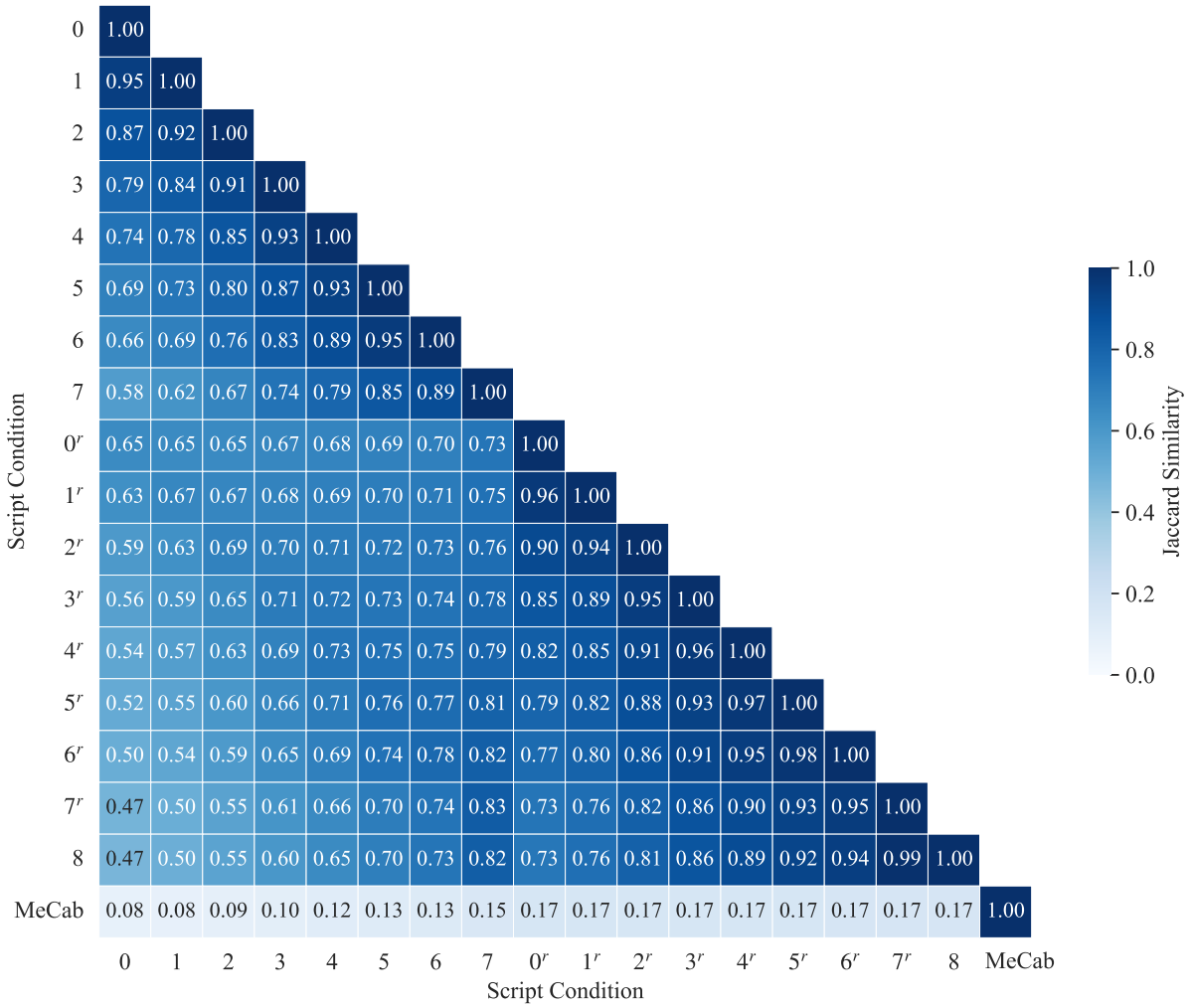


Figure 3: Vocabulary overlap between each pair of script-transformed development sets. Overlap is calculated as the Jaccard similarity between the sets of unique BPE tokens appearing in each dataset.

Parameter	Value
attention dropout	0.1
hidden_size	768
num_attention_heads	12
num_hidden_layers	12
embedding_dim	768
vocab_size	32,000
block size	512
optimizer	AdamW
learning rate	5e-5
learning rate scheduler	constant with warmup
warmup steps	5,000
batch size	64
seed	1,2,3
epoch	10

Table 9: Hyperparameters used for model pre-training.

C.2 Implementation

For preprocessing, we use MeCab (Kudo, 2005) (`v0.996`) with UniDic 3.1.0 (Den et al., 2007) using the default parameters. We train a BPE tokenizer using Hugging Face's Tokenizers (Moi and Patry, 2023) (`v0.22.2`), with NFKC normalization and a vocabulary size of 32,000, trained jointly on the original corpus and its fully kana-transliterated counterpart. We use Hugging Face's Transformers (Wolf et al., 2020) (`v4.57.6`) for pre-training.

D License

In Table 10, we list the licenses of the data and tools used in this work. All resources were used in accordance with their respective licenses.

Data/tool	License
ReazonSpeech (Yin et al., 2023)	CDLA-Sharing-1.0
MeCab (Kudo, 2005)	BSD, GPL or LGPL
UniDic 3.1.0 (Den et al., 2007)	BSD, GPL or LGPL
Tokenizers (Moi and Patry, 2023)	Apache 2.0
Transformers (Wolf et al., 2020)	Apache 2.0

Table 10: License of the data and tools.

E Additional Experimental Results

E.1 Perplexity Results

Table 11 reports test perplexity across training schedules and evaluation script levels. To examine whether language modeling quality (perplexity) correlates with syntactic generalization (JBLiMP), we computed Spearman's rank correlation between test perplexity and JBLiMP accuracy across all training schedules for each evaluation script level. We ranked the training schedules by ascending test perplexity and descending JBLiMP accuracy, such that lower ranks indicated better performance for both measures. We then computed Spearman's rank correlation between these rankings at each evaluation script level. The correlation was strong across evaluation conditions ($\rho \in [0.73, 0.96]$), indicating that schedules with lower perplexity generally achieve higher JBLiMP scores regardless of the evaluation script. The strong correlation between perplexity and JBLiMP accuracy across evaluation levels suggests that the observed trends are not specific to the JBLiMP benchmark, but also reflect intrinsic language modeling performance.

E.2 Additional JBLiMP Results

Table 12 presents the JBLiMP accuracy results across all pre-training schedules and script variants, including both kana-transliterated and ruby-annotated training conditions. Compared with the main results in Table 7, this table additionally includes all ruby-based transition schedules and their evaluation performance across different script conditions. Overall, ruby-based schedules tend to underperform their kana-transliterated counterparts, likely because of the additional sequence complexity introduced by markup annotations. However, when evaluated on the original JBLiMP benchmark (L=8), ruby-based schedules often achieve slightly stronger performance, with a few exceptions, suggesting that preserving original kanji surface forms during training may provide useful orthographic information.

E.3 Recency Effect

To better understand the robustness of the randomized schedule and the role of recency, we conducted additional experiments where the first nine training epochs were sampled randomly from the original (L=8) and fully kana-transliterated (L=0) corpora, while fixing the final epoch to a single script variant (L=0, ..., L=8). This design isolates the influence of the final training epoch while preserving the same random exposure throughout the preceding training.

Table 13 shows the JBLiMP accuracies across different script conditions. We observe recency effects at the lower-level kana-transliterated variants (L=0, L=1, L=2, L=3), where the models trained on simpler scripts in the final epoch perform better than the fully randomized schedule; however, the fully randomized schedule still achieves the highest average accuracy across evaluation conditions. While recent exposure facilitates adaptation to a specific script condition, continual exposure to mixed scripts is more effective for learning representations that generalize across script variants.

E.4 Additional Learning Trajectories

Figure 4 shows JBLiMP accuracy trajectories across training epochs for the forward and backward transition schedules, respectively, evaluated on both the original benchmark (L=8) and the fully kana-transliterated benchmark (L=0). Overall, the trajectories indicate that performance improves substantially once models are exposed to training data matching the evaluation script condition. For example, forward schedules rapidly improve on the original JBLiMP benchmark after transitioning to raw text, whereas backward schedules steadily improve on the kana-transliterated benchmark once kana-only training is introduced. However, these improvements generally come at the expense of reduced performance on the opposite script condition, indicating a trade-off between adaptation to a specific script variant and robustness across multiple script conditions.

Schedule	Test Perplexity									
	L=8	L=7	L=6	L=5	L=4	L=3	L=2	L=1	L=0	AVG
8 8 8 8 8 8 8 8 8 8	114.5	342.7	602.9	818.4	1,231.6	1,763.5	2,458.1	3,255.2	3,687.9	1,586.1
0 0 0 0 0 0 0 0 0 0	199,328.4	40,718.0	21,003.3	14,968.0	8,330.1	4,082.8	1,360.6	347.6	135.0	32,252.7
0^r 0^r 0^r 0^r 0^r 0^r 0^r 0^r 0^r 0^r	112,915.9	37,687.1	27,464.3	23,410.2	17,711.9	12,553.3	7,625.2	4,222.0	3,052.0	27,404.7
0 0 0 0 0 0 0 0 0 0 8	183.1	263.4	314.0	345.3	389.7	433.8	469.1	507.7	513.8	380.0
0 0 0 0 0 0 0 0 0 8 8	146.7	243.3	310.3	352.8	417.0	483.5	541.0	595.1	602.6	410.2
0 0 0 0 0 0 0 0 8 8 8	134.2	242.7	323.0	375.3	456.2	541.0	616.1	681.1	685.8	450.6
0 0 0 0 0 0 0 8 8 8 8	127.7	247.3	341.1	404.2	503.6	609.1	704.3	782.3	785.9	500.6
0 0 0 0 0 0 8 8 8 8 8	123.3	253.3	360.7	434.0	552.0	678.8	796.5	892.7	897.3	554.3
0 0 0 0 0 8 8 8 8 8 8	120.4	262.3	385.5	471.6	612.5	766.0	913.9	1,037.2	1,052.5	624.7
0 0 0 0 8 8 8 8 8 8 8	118.3	273.2	414.9	516.1	684.6	868.6	1,048.2	1,197.7	1,218.2	704.4
0 0 0 8 8 8 8 8 8 8 8	117.3	290.6	458.3	581.0	791.6	1,028.6	1,267.3	1,473.1	1,510.2	835.3
0 0 8 8 8 8 8 8 8 8 8	116.6	316.0	522.3	679.7	961.2	1,292.7	1,653.1	1,984.8	2,075.5	1,066.9
8 0 0 0 0 0 0 0 0 0 0	155,334.5	27,226.2	13,618.9	9,508.2	5,259.2	2,637.3	967.4	296.4	136.7	23,887.2
8 8 0 0 0 0 0 0 0 0 0	196,378.3	30,585.3	14,498.5	9,810.5	5,191.4	2,534.4	911.0	288.0	137.4	28,926.1
8 8 8 0 0 0 0 0 0 0 0	57,180.9	12,056.2	6,592.1	4,817.4	2,873.0	1,580.0	676.2	257.7	138.2	9,574.7
8 8 8 8 0 0 0 0 0 0 0	20,704.1	5,561.8	3,373.0	2,611.3	1,722.0	1,049.4	524.9	234.7	139.7	3,991.2
8 8 8 8 8 0 0 0 0 0 0	9,970.3	3,227.8	2,115.2	1,709.0	1,209.4	793.9	441.9	221.4	141.5	2,203.4
8 8 8 8 8 8 0 0 0 0 0	5,133.3	2,028.1	1,434.7	1,205.9	907.1	637.1	387.2	214.3	145.4	1,343.7
8 8 8 8 8 8 8 0 0 0 0	2,894.5	1,359.2	1,032.4	899.1	713.3	532.3	350.2	211.9	151.7	904.9
8 8 8 8 8 8 8 8 0 0 0	1,749.8	970.1	789.4	711.5	593.3	468.6	331.5	217.4	163.4	666.1
8 8 8 8 8 8 8 8 8 0 0	1,070.1	714.5	629.4	589.6	519.9	437.8	335.5	239.8	189.4	525.1
Random(8, 0)^{10}	140.7	213.6	258.6	283.8	308.7	314.2	278.7	213.9	164.0	241.8
0 1 2 3 4 5 6 7 8 8	125.2	200.9	260.1	305.2	396.7	523.0	709.8	941.1	1,113.0	508.3
8 8 7 6 5 4 3 2 1 0	9,211.2	1,792.2	978.3	743.8	520.5	363.9	244.4	167.6	143.2	1,573.9

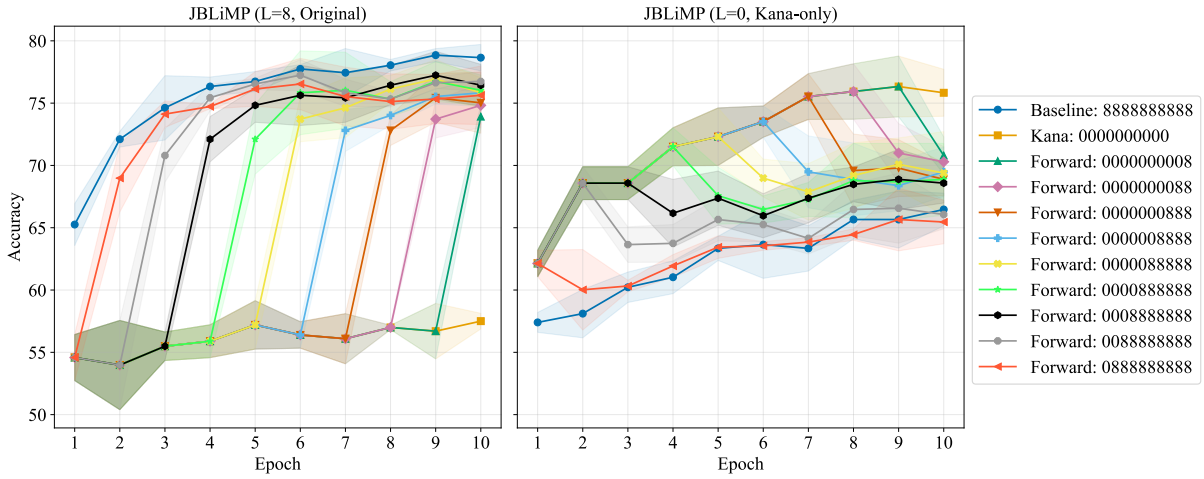
Table 11: Test perplexity results across script variants and pre-training schedules. Columns (L=8 $\rightarrow$ L=0) denote script-transformed evaluation sets with progressively reduced kanji inventories, where L=8 denotes the original benchmark (raw) and L=0 corresponds to fully kana-transliterated text. AVG denotes the macro-average perplexity across all evaluation levels.

Script	Type	Schedule	JBLiMP (Average Accuracy)									
			L=8	L=7	L=6	L=5	L=4	L=3	L=2	L=1	L=0	AVG
Single-level	Baseline	8 8 8 8 8 8 8 8 8 8	**78.7**	74.2	71.1	71.0	70.6	70.2	68.0	68.1	66.5	70.9
	Kana	0 0 0 0 0 0 0 0 0 0	57.5	60.8	61.3	63.2	63.5	63.2	65.8	69.5	75.8	64.5
	Ruby	$0^r0^r0^r0^r0^r0^r0^r0^r0^r0^r$	54.3	58.2	57.3	59.2	59.3	57.9	58.9	62.2	65.1	59.2
Two-level	Forward	0 0 0 0 0 0 0 0 0 8	73.9	73.2	72.3	72.1	72.9	73.9	73.4	72.8	70.8	72.8
		0 0 0 0 0 0 0 0 8 8	74.8	74.7	73.3	73.7	74.0	75.1	74.1	73.9	70.3	73.8
		0 0 0 0 0 0 0 8 8 8	75.0	73.7	71.7	71.9	72.5	73.1	72.2	72.0	68.9	72.3
		0 0 0 0 0 0 8 8 8 8	75.8	74.7	71.9	72.0	72.4	71.8	70.4	71.4	69.5	72.2
		0 0 0 0 0 8 8 8 8 8	75.8	73.5	71.2	71.1	71.5	71.6	71.0	72.1	69.4	71.9
		0 0 0 0 8 8 8 8 8 8	76.0	74.3	71.6	70.9	71.2	70.1	68.5	69.8	68.9	71.3
		0 0 0 8 8 8 8 8 8 8	76.4	74.7	72.4	72.2	72.2	70.4	68.9	69.3	68.6	71.7
		0 0 8 8 8 8 8 8 8 8	76.7	74.2	71.4	71.3	71.1	69.5	67.7	68.1	66.1	70.7
		0 8 8 8 8 8 8 8 8 8	75.6	74.1	71.8	71.5	71.0	69.7	66.5	67.1	65.5	70.3
		$0^r0^r0^r0^r0^r0^r0^r0^r0^r8$	74.9	71.8	68.0	68.2	68.2	67.2	65.6	67.7	64.6	68.4
		$0^r0^r0^r0^r0^r0^r0^r0^r8$ 8	76.7	72.0	68.2	68.3	68.4	68.8	66.6	67.7	64.9	69.1
		$0^r0^r0^r0^r0^r0^r0^r8$ 8 8	77.5	72.1	68.2	68.5	68.3	68.4	66.6	67.5	66.2	69.2
		$0^r0^r0^r0^r0^r0^r8$ 8 8 8	76.3	72.4	69.7	69.6	69.1	68.5	66.8	67.9	65.6	69.5
		$0^r0^r0^r0^r0^r8$ 8 8 8 8	76.9	73.6	69.3	69.6	69.0	67.9	65.6	66.5	66.1	69.4
		$0^r0^r0^r0^r8$ 8 8 8 8 8	76.4	73.3	70.4	70.7	69.7	68.6	66.4	66.9	63.6	69.6
		$0^r0^r0^r8$ 8 8 8 8 8 8	75.9	73.4	69.7	69.8	69.8	68.5	65.0	66.6	64.4	69.2
		0^r0^r8 8 8 8 8 8 8 8	77.2	73.2	69.2	69.1	68.7	67.6	64.8	66.4	64.5	68.9
		0^r8 8 8 8 8 8 8 8 8	76.5	73.0	69.9	70.0	69.2	68.6	65.6	67.7	64.5	69.4
	Backward	8 0 0 0 0 0 0 0 0 0	56.4	59.0	59.8	62.6	63.1	63.0	65.1	68.8	74.4	63.6
		8 8 0 0 0 0 0 0 0 0	55.3	61.0	61.8	64.4	64.7	64.2	66.8	69.9	75.0	64.8
		8 8 8 0 0 0 0 0 0 0	60.0	66.2	66.6	69.0	69.2	68.5	69.4	72.9	76.5	68.7
		8 8 8 8 0 0 0 0 0 0	66.2	67.3	67.6	70.0	70.1	70.1	71.1	73.7	**77.0**	70.3
		8 8 8 8 8 0 0 0 0 0	66.8	68.7	68.4	69.6	69.9	70.1	70.8	73.1	76.5	70.4
		8 8 8 8 8 8 0 0 0 0	70.6	70.0	69.4	70.9	71.0	71.4	72.0	73.4	76.4	71.7
		8 8 8 8 8 8 8 0 0 0	72.2	71.4	70.7	72.5	72.5	72.4	72.5	74.0	75.2	72.6
		8 8 8 8 8 8 8 8 0 0	72.9	72.3	71.1	71.6	71.4	72.1	73.3	74.0	75.3	72.7
		8 8 8 8 8 8 8 8 8 0	71.1	71.7	71.5	71.8	71.6	71.4	72.3	72.9	74.6	72.1
		8 $0^r0^r0^r0^r0^r0^r0^r0^r0^r$	57.1	58.1	56.8	58.3	59.0	58.0	60.3	63.8	65.4	59.7
		8 8 $0^r0^r0^r0^r0^r0^r0^r0^r$	60.7	60.7	58.3	59.8	59.9	59.0	59.3	62.1	63.3	60.4
		8 8 8 $0^r0^r0^r0^r0^r0^r0^r$	63.1	61.9	58.6	60.0	60.5	60.0	60.5	62.9	63.7	61.3
		8 8 8 8 $0^r0^r0^r0^r0^r0^r$	61.8	61.8	59.3	59.8	59.7	59.6	60.5	63.6	64.4	61.2
		8 8 8 8 8 $0^r0^r0^r0^r0^r$	63.9	64.0	61.3	62.1	61.8	61.1	60.9	63.5	65.5	62.7
		8 8 8 8 8 8 $0^r0^r0^r0^r$	66.0	63.5	60.5	61.4	61.5	62.1	63.0	65.5	66.3	63.3
		8 8 8 8 8 8 8 $0^r0^r0^r$	66.4	64.7	61.9	63.4	63.2	62.5	62.2	64.5	65.0	63.8
		8 8 8 8 8 8 8 8 0^r0^r	68.4	66.2	63.4	63.6	64.2	64.0	63.8	65.1	63.6	64.7
		8 8 8 8 8 8 8 8 8 0^r	68.9	66.2	64.0	64.7	64.2	64.0	63.2	63.7	63.2	64.7
	Random	$\text{Random}(8, 0)^{10}$	76.2	**77.1**	**76.1**	**75.7**	**75.6**	**76.7**	**75.8**	**75.6**	75.5	**76.1**
		$\text{Random}(8, 0^r)^{10}$	78.4	73.2	71.3	70.9	70.5	69.4	67.5	67.5	65.1	70.4
Multi-level	Progressive	0 1 2 3 4 5 6 7 8 8	76.3	73.5	72.4	73.0	73.4	73.0	71.3	72.0	67.8	72.5
		$0^r1^r2^r3^r4^r5^r6^r7^r8$ 8	76.9	73.9	70.0	69.9	69.6	68.8	65.9	66.6	66.4	69.8
	Reverse	8 8 7 6 5 4 3 2 1 0	67.2	71.2	71.6	72.9	73.0	74.0	74.5	75.2	75.9	72.8
		8 8 $7^r6^r5^r4^r3^r2^r1^r0^r$	69.5	69.8	65.5	66.9	67.2	66.0	67.0	66.5	66.2	67.1

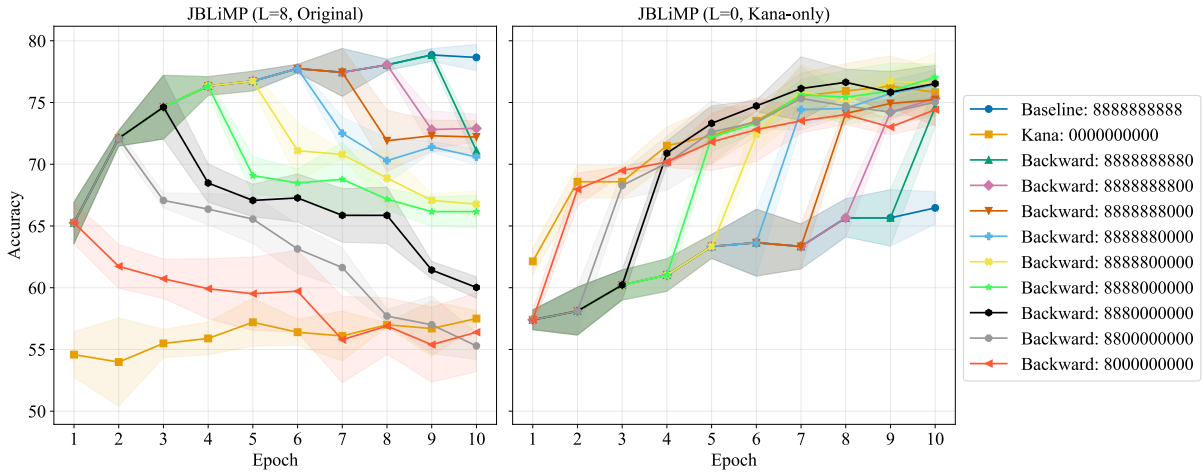
Table 12: Main JBLiMP accuracy results across script variants and pre-training schedules (averaged across three runs). Columns (L=8 → L=0) denote script-transformed evaluation sets with progressively reduced kanji inventories, where L=8 denotes the original benchmark (raw) and L=0 corresponds to full kana-transliteration. AVG denotes the macro-average accuracy across all evaluation levels. Boldface indicates the best-performing model for each evaluation condition.

Schedule	JBLiMP (Average Accuracy)									
	L=8	L=7	L=6	L=5	L=4	L=3	L=2	L=1	L=0	AVG
Random(8, 0)^{10}	**76.2**	**77.1**	76.1	75.7	75.6	76.7	75.8	75.6	75.5	**76.1**
Random(8, 0)^{9}, 8	75.9	76.0	**76.5**	**77.2**	**77.0**	76.3	75.3	76.1	74.3	**76.1**
Random(8, 0)^{9}, 7	76.0	76.1	74.7	75.2	75.3	76.3	75.3	75.6	74.3	75.5
Random(8, 0)^{9}, 6	75.0	75.7	75.7	76.1	76.2	**77.4**	76.1	76.2	74.3	75.9
Random(8, 0)^{9}, 5	75.5	75.2	74.6	75.0	74.7	75.5	74.0	74.4	73.1	74.7
Random(8, 0)^{9}, 4	74.8	75.2	74.8	75.3	75.6	76.5	74.2	74.4	73.0	74.9
Random(8, 0)^{9}, 3	74.4	75.5	74.9	75.6	75.9	77.1	75.8	75.7	74.2	75.5
Random(8, 0)^{9}, 2	73.8	75.2	75.4	75.5	75.0	75.9	**76.7**	76.5	75.3	75.5
Random(8, 0)^{9}, 1	73.1	72.7	73.3	73.4	73.0	74.9	75.8	**78.0**	76.0	74.5
Random(8, 0)^{9}, 0	72.4	72.1	72.9	72.8	72.6	73.3	72.9	73.9	**76.1**	73.2

Table 13: JBLiMP accuracy under randomized schedules with different script variants fixed for the final training epoch. Results are averaged across three runs. Random(8, 0)^{10} uses randomized exposure throughout all ten epochs. Random(8, 0)^{9}, L denotes nine epochs of randomized exposure to levels 8 and 0, followed by one epoch at level L. AVG reports the average accuracy across the nine evaluation conditions. The best result in each column is shown in bold.



(a) Forward transition schedules.



(b) Backward transition schedules.

Figure 4: Learning trajectories across training epochs for forward and backward transition schedules, evaluated on the original JBLiMP benchmark (L=8) and the fully kana-transliterated benchmark (L=0). We report accuracy at each training epoch.